Experimental Report on the School of Computer and Artificial Intelligence at Zhengzhou University

Grade：2024 Major：SOFTWARE ENGINEERING Class：GROUP-1

Name：MD TASNIMUL HASAN Student Number：202480090104

Instructor：ALEX Score： 2026 year 10 month 16 date

Experimental Topic：Menus, Lists, and Dialog Boxes

1. Experimental Purpose

- Master the creation and use of Android menus (Options Menu, Context Menu, Popup Menu).
- Master the implementation of Android lists using ListView with adapters (e.g., ArrayAdapter, custom adapters).
- Master the design and display of Android dialog boxes (AlertDialog, custom dialogs, DatePickerDialog, etc.).
- Understand and apply Toast for lightweight user feedback and Notification for status-bar notifications.

2. Experimental Content

1.Menus

- Create an Options Menu with several items (e.g., Add, Delete, Settings, About).
- Implement a Context Menu triggered by a long-press on a list item.
- Implement a Popup Menu triggered by a button click.

2.Lists

- Display a list of items (e.g., student names or courses) using ListView.
- Use an ArrayAdapter for simple text items and a custom adapter for items with icon + text.

- Handle item click events (e.g., show a Toast with the selected item).

3.Dialog Boxes

- Show an AlertDialog with OK/Cancel buttons (e.g., delete confirmation).
- Show a custom dialog containing an EditText to add a new list item.
- (Optional) Show a DatePickerDialog or TimePickerDialog.

4.Toast and Notification

- Use Toast to give short feedback on user actions.
- Create and send a Notification (with channel on Android 8.0+) when, for example, an item is added.

3. Experimental Environment

Item	Specification
OS	Windows 10 / 11 (or macOS)
IDE	Android Studio (Giraffe / Hedgehog or later)
Language	Java (or Kotlin)
SDK	Android SDK API 24 – 34 (minSdk 24, targetSdk 34)
Emulator/Device	Android Emulator (Pixel, API 30+) or physical device
Build System	Gradle

4. Procedure

Step 1: Create the Project

1.Open Android Studio and create a **New Project** using the **Empty Views Activity** template.

2.Set the project name to MenuListDialogDemo, choose **Java** as the language, and set the minimum SDK to **API 24**.

3.Wait for the Gradle sync to complete successfully.

Step 2: Design the Main Layout

1.Open activity_main.xml.

2.Add a vertical LinearLayout as the root container with some padding.

3.Inside it, place:

- A **Button** (id: btnAdd) labeled "Add Item (Dialog)".
- A **Button** (id: btnPopup) labeled "Show Popup Menu".
- A **ListView** (id: listView) that fills the remaining space using layout_weight="1".

4.This layout provides the trigger buttons for dialogs and popup menus, and the list area for displaying items.

Step 3: Define the Options Menu Resource

1.Create a new resource file main_menu.xml inside the res/menu/ folder.

2.Inside the <menu> tag, add four <item> entries with IDs menu_add, menu_delete, menu_settings, and menu_about, each with a corresponding title.

3.This XML file declares the Options Menu that will appear in the action bar.

Step 4: Implement the Main Activity Logic

1.Open MainActivity.java and declare member variables for the ListView, the ArrayAdapter, and the data list (an ArrayList of strings).

2.In onCreate():

- Inflate the layout, bind the ListView, and initialize the adapter with the data list, then attach the adapter to the list.
- Create a notification channel (required for Android 8.0 and above).
- Register an **item-click listener** on the list so that tapping an item shows a **Toast** with the item name.
- Register the ListView for a **context menu** so that long-pressing an item opens a context menu.
- Attach click listeners to the two buttons: the Add button opens the custom input dialog, and the Popup button shows a popup menu.

3.Override onCreateOptionsMenu() to inflate main_menu.xml into the Options Menu.

4.Override onOptionsItemSelected() to handle each menu item:

- "Add" → open the custom add dialog.
- "Delete" → open a delete-confirmation AlertDialog.

- "About" → show an information AlertDialog.
- "Settings" → show a Toast.

5.Override onCreateContextMenu() to define two context-menu options ("Delete Item" and "Show Info") with a header title.

6.Override onContextItemSelected() to:

- Remove the long-pressed item from the list and refresh the adapter (Delete).
- Show a Toast with the item's content (Info).

Step 5: Implement the Popup Menu

1.Create a helper method showPopupMenu(View anchor).

2.Inside it, instantiate a PopupMenu anchored to the clicked button, inflate main_menu.xml into it, reuse the same onOptionsItemSelected() handler, and call show().

Step 6: Implement the Custom Add Dialog

1.Create a helper method showAddDialog().

2.Inflate a small custom layout (dialog_add.xml) containing a single EditText.

3.Build an AlertDialog using AlertDialog.Builder with:

- A title "Add New Item".
- The custom view set via setView().
- A positive "Add" button that reads the text from the EditText, adds it to the data list (if non-empty), refreshes the adapter, shows a Toast, and sends a Notification.
- A negative "Cancel" button that does nothing.

Step 7: Implement the Delete Confirmation Dialog

1.Create a helper method showDeleteDialog().

2.Build an AlertDialog with a title "Delete All?", a message, a positive "Yes" button that clears the data list and refreshes the adapter, and a negative "No" button that dismisses the dialog.

Step 8: Implement Toast and Notification

1. **Toast**: Use Toast.makeText(...).show() wherever short feedback is needed (item clicks, additions, deletions, settings).

2. **Notification Channel**: In a helper method createNotificationChannel(), check if the API level is 26 or higher, and if so, create a NotificationChannel with an ID, a name, and default importance, then register it with the NotificationManager.
3. **Send Notification**: In a helper method sendNotification(String text), build a NotificationCompat.Builder with a small icon, title, and content text, then post it using NotificationManagerCompat.notify() with a unique ID.

Step 9: Create the Custom Dialog Layout

1. Create dialog_add.xml inside res/layout/.
2. Add a LinearLayout containing a single EditText (id: etItem) with a hint "Enter item name".
3. This layout is used as the custom view inside the add-item dialog.

Step 10: Run and Test the Application

1. Click **Run** to build and install the app on the emulator or a physical device.
2. Verify each feature step by step:

- Open the **Options Menu** from the action bar and test each item.
- **Long-press** a list item to open the Context Menu and test Delete/Info.
- Click **Show Popup Menu** and verify the popup appears anchored to the button.
- Click **Add Item**, enter text, confirm, and check that the list updates, a Toast appears, and a Notification is posted.
- Tap a list item and confirm the Toast shows the correct name.
- Open the status bar and verify the Notification is visible.

3.If a feature does not work (e.g., no notification appears), check logcat, verify the notification channel was created, and confirm the required permissions are granted on Android 13+.

5. Results and Analysis

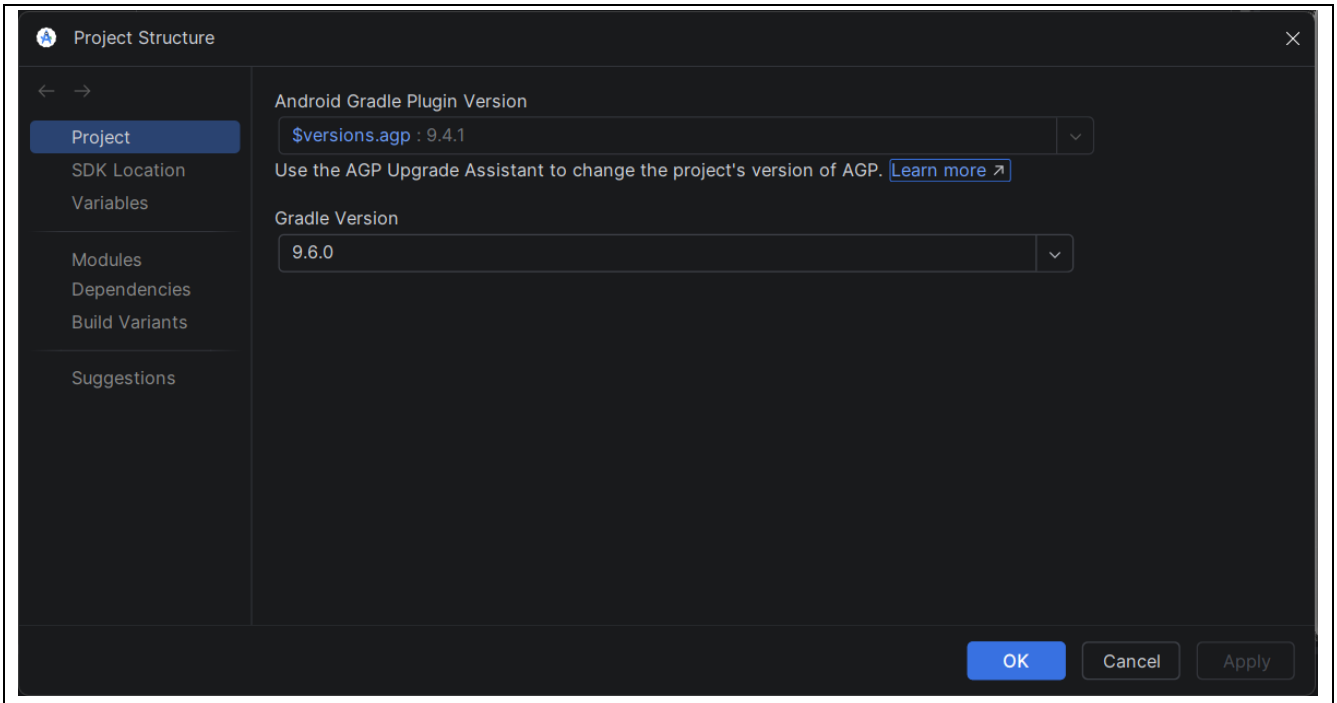


Figure 1: Project Structure

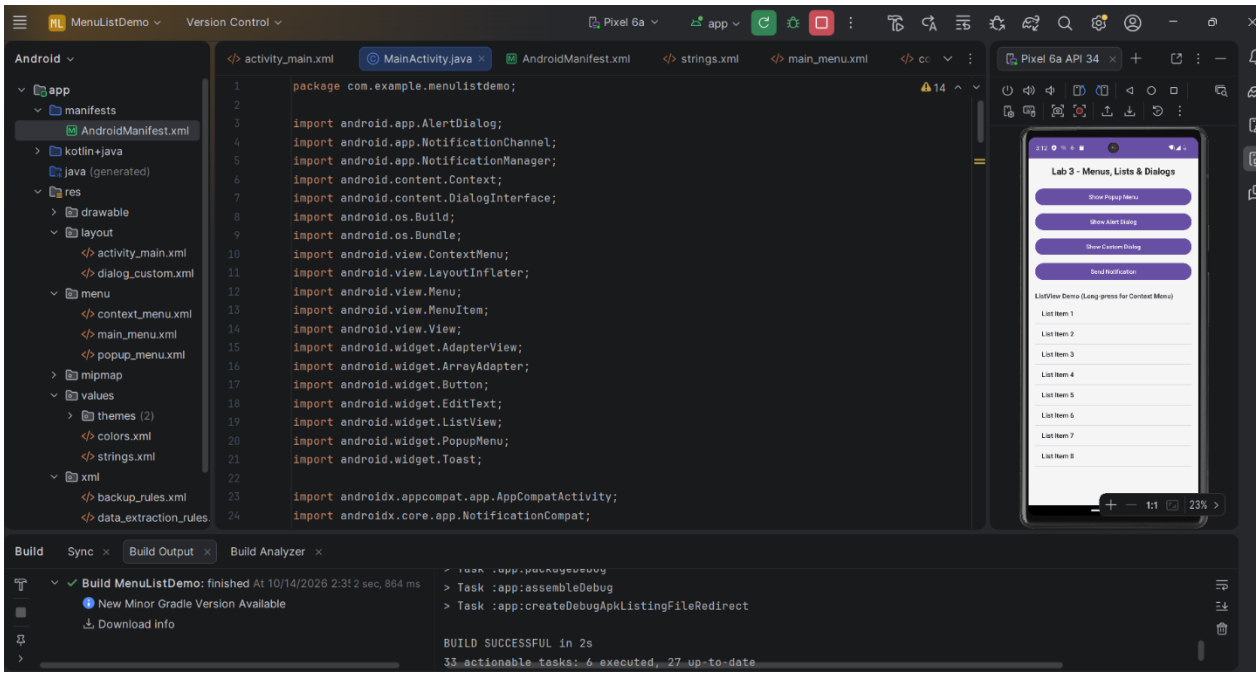


Figure 2: MainActivity.java Code

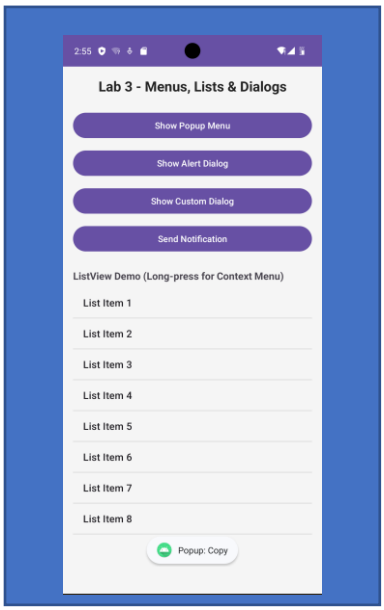


Figure 3: Popup Menu (Copy)

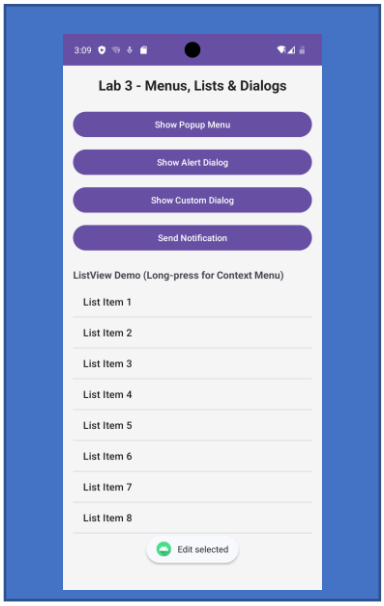


Figure 4: Context Menu (Long-press)

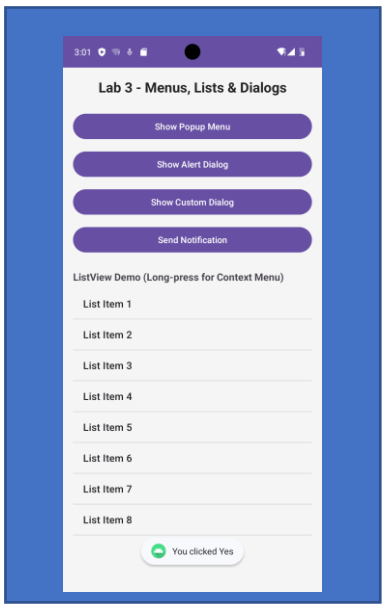


Figure 5: Alert Dialog (Yes)

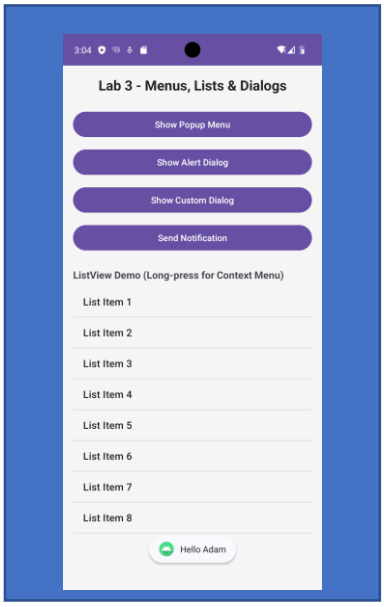


Figure 6: Custom Dialog with EditText

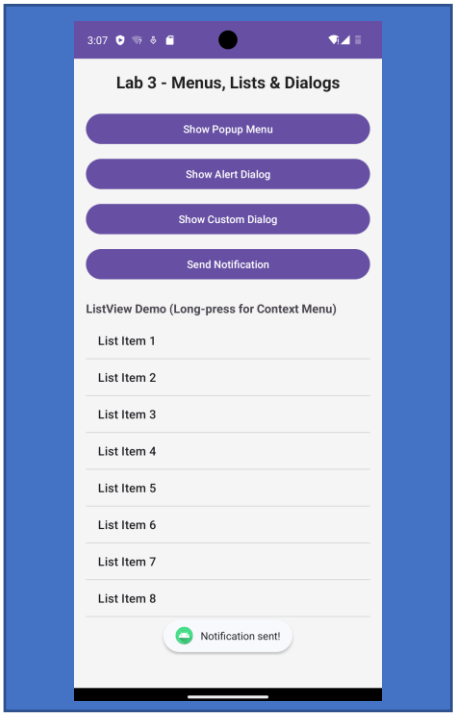


Figure 7: Notification in Status Bar

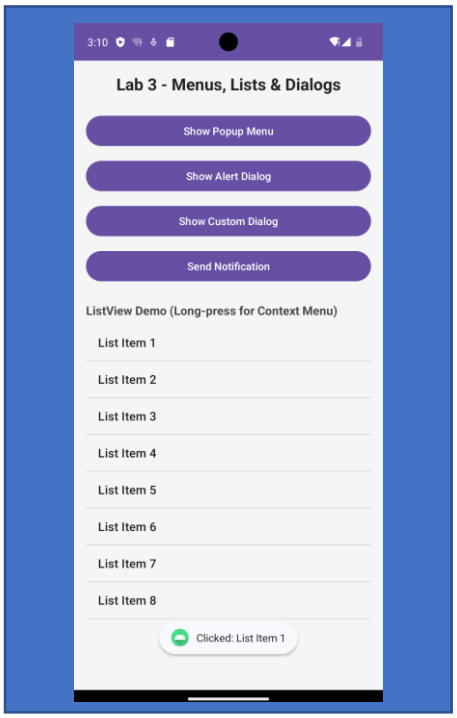


Figure 8: ListView Item Click

Results

- The Options Menu correctly displayed all four items and responded to each selection.
- The Context Menu appeared on long-press of a list item and allowed deletion/info display of the selected item.
- The Popup Menu anchored to the button and reused the options-menu handler.
- The custom AlertDialog accepted user input, added items to the ListView, and refreshed the display via notifyDataSetChanged().
- The delete-confirmation AlertDialog cleared the list only on positive confirmation.
- Toast messages confirmed all user actions.
- Notification appeared in the status bar on Android 8.0+ (required NotificationChannel).

Analysis

- ListView + ArrayAdapter is adequate for simple text data, but a custom adapter is needed for icons or multi-line items.

- Menu resources (res/menu/*.xml) separate UI definition from logic, improving maintainability.
- AlertDialog.Builder provides a fluent API for standard dialogs; setView() enables custom layouts.
- For Android 8.0+ (API 26), notifications **must** be assigned to a NotificationChannel, otherwise they are silently dropped.
- On Android 13+ (API 33), the POST_NOTIFICATIONS runtime permission must be requested for notifications to appear.
- Updating list data requires calling notifyDataSetChanged() (or using DiffUtil) so the UI reflects changes.
-

6. Summary

In this experiment, I practiced the core Android UI components for menus, lists, and dialogs:

- **Menus** — Implemented three menu types: Options Menu (action-bar), Context Menu (long-press), and Popup Menu (anchor view). Learned to define menus declaratively in XML and handle selections in callbacks.
- **Lists** — Used ListView with ArrayAdapter to display and dynamically update a collection of strings; handled item-click and context-menu events.
- **Dialogs** — Used AlertDialog for confirmations and custom-view dialogs for text input, and understood the Builder pattern.
- **Toast & Notification** — Used Toast for transient feedback and built a channel-based Notification for persistent status-bar alerts.

Key takeaways: keep resources (menus/layouts) separate from code; always refresh adapters after data changes; respect API-level requirements for channels and notification permissions. The experiment strengthened my ability to combine multiple UI components into a coherent, interactive Android application.